

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Debugging & Optimization Task

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 2 – Debugging & Optimisation Task
Weightage:	35% of CIA	Total Marks:	100
CO2 Statement:	<i>Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.</i>		
Student Name:	G.KOUSHIK	Reg. No.:	192524238
Section / Batch:	BTECH AIDS	Date:	22/07/2026

Code of Conduct

I G.KOUSHIK(192524238)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:G.KOUSHIK

Instructions

- This test contains 20 Debugging & Optimisation Task questions. Total: 100 Marks (20*5).
- When a student answer using Scenario-Based, in evaluation the answer should be with following five requirements: Identify the problem type and constraints, Select appropriate data structure, Design the Solution, Optimization and efficiency, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Search Data Structure, Stack	Binary Search, Stack Operations	1 - 5	25
Linked List Data Structure	List Operations	6 - 10	25
Queue Data Structures	Queue Operations	11 - 15	25
Queue Data Structures, Stack Notations	Queue Operations, Expression Evaluation	16 - 20	25
GRAND TOTAL:			100 Marks

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Debugging Accuracy	20	All errors corrected; code runs flawlessly	Most errors corrected; minor issues remain	Partial correction; code runs with issues	Code fails or major errors persist
Error Identification	30	Accurately identifies all logical, syntax, and edge-case errors	Identifies most major errors	Identifies some errors but misses key issues	Struggles to identify errors
Code Quality & Readability	20	Clean, modular, well-documented, follows best practices	Mostly clean and readable	Some structure but inconsistent	Poorly written, hard to understand
Explanation & Justification	20	Clear, logical, and well-justified reasoning with complexity analysis	Good explanation with some justification	Limited explanation; lacks depth	No or incorrect explanation
Testing & Validation	10	Comprehensive test cases including edge cases	Adequate test cases	Minimal testing	No testing provided

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SIMATS Online Programming Lab

DS PROGRAMMING



GANGINENI Koushik
192524238RB



🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int stack[MAX];
6 int top = MAX - 1;
7
8 int main() {
9     if (top == MAX - 1)
10         printf("Overflow");
11     else
12         printf("Space available");
13
14     return 0;
15 }
```

Test Input

stdin input (optional)

Output

Overflow



Pending manual review



🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 int stack[5] = {10, 20, 30};
4 int top = 2;
5
6 int pop() {
7     if (top == -1) {
8         printf("Underflow");
9         return -1;
10    }
11
12    return stack[top--];
13 }
14
15 int main() {
16     printf("Popped element = %d", pop());
17     return 0;
18 }
```

Test Input

stdin input (optional)

Output

Popped element = 30

🕒 Pending manual review



🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *prev;
7     struct Node *next;
8 };
9
10 void deleteNode(struct Node **head, struct Node *del) {
11     if (del == NULL)
12         return;
13
14     if (*head == del)
15         *head = del->next;
16
17     if (del->prev != NULL)
18         del->prev->next = del->next;
19
20     if (del->next != NULL)
21         del->next->prev = del->prev;
22
23     free(del);
24 }
```

Test Input

Number of nodes: 3
Elements: 10 20 30
Node to delete: 20

Output

Node deletion operation completed.

🕒 Pending manual review



Your Fixed Code

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int queue[MAX];
6 int rear = 0;
7
8 void enqueue(int data) {
9     if (rear == MAX) {
10         printf("Queue Full\n");
11         return;
12     }
13
14     queue[rear++] = data;
15 }
16
17 int main() {
18     enqueue(10);
19     enqueue(20);
20
21     printf("Queue: %d %d", queue[0], queue[1]);
22
23     return 0;
24 }
```

Test Input

10 20

Output

Queue: 10 20

⌚ Pending manual review



Your Fixed Code

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int deque[MAX];
6 int front = 2, rear = 2;
7
8 void insertFront(int data) {
9     if ((front == 0 && rear == MAX - 1) ||
10         front == rear + 1) {
11         printf("Deque Full");
12         return;
13     }
14
15     if (front == 0)
16         front = MAX - 1;
17     else
18         front = front - 1;
19
20     deque[front] = data;
21 }
22
23 int main() {
24     insertFront(10);
25     printf("Inserted element = %d", deque[front]);
26
27     return 0;
28 }
```

Test Input

front = 2
data = 10

Output

Inserted element = 10

⌚ Pending manual review

Your Fixed Code

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int deque[MAX];
6 int front = 2, rear = 2;
7
8 void insertFront(int data) {
9     if ((front == 0 && rear == MAX - 1) ||
10         front == rear + 1) {
11         printf("Deque Full");
12         return;
13     }
14
15     if (front == 0)
16         front = MAX - 1;
17     else
18         front = front - 1;
19
20     deque[front] = data;
21 }
22
23 int main() {
24     insertFront(10);
25     printf("Inserted element = %d", deque[front]);
26
27     return 0;
```

Test Input

```
front = 2
data = 10
```

Output

```
Inserted element = 10
```

 Pending manual review

```
2
3 int binarySearch(int arr[], int l, int r, int x) {
4     if (l > r)
5         return -1;
6
7     int mid = l + (r - l) / 2;
8
9     if (arr[mid] == x)
10        return mid;
11
12    if (arr[mid] > x)
13        return binarySearch(arr, l, mid - 1, x);
14
15    return binarySearch(arr, mid + 1, r, x);
16 }
17
18 int main() {
19     int arr[] = {10, 20, 30, 40, 50};
20     int x = 40;
21
22     int result = binarySearch(arr, 0, 4, x);
23
24     if (result != -1)
25         printf("Element found at index %d", result);
26     else
27         printf("Element not found");
28
29     return 0;
30 }
```

```
Key: 40
```

Output

```
Element found at index 3
```

 Pending manual review



Your Fixed Code

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int queue[MAX] = {10, 20, 30};
6 int front = 0, rear = 2;
7
8 int dequeue() {
9     if (front == -1) {
10         printf("Underflow\n");
11         return -1;
12     }
13
14     int data = queue[front];
15
16     if (front == rear)
17         front = rear = -1;
18     else
19         front = (front + 1) % MAX;
20
21     return data;
22 }
23
24 int main() {
25     int x = dequeue();
26
27     if (x != -1)
28         printf("Deleted element = %d", x);
```

Test Input

```
Queue = {10, 20, 30}
front = 0
rear = 2
```

Output

```
Deleted element = 10
```

Pending manual review



Your Fixed Code

```
1 #include <stdio.h>
2
3 int stack[10], top = -1;
4
5 void push(int x) {
6     stack[++top] = x;
7 }
8
9 int pop() {
10     return stack[top--];
11 }
12
13 int main() {
14     char expression[] = "23+";
15     int i = 0, result;
16
17     while (expression[i] != '\0') {
18         if (expression[i] >= '0' && expression[i] <= '9')
19             push(expression[i] - '0');
20         else {
21             int op2 = pop();
22             int op1 = pop();
23             result = op1 + op2;
24             push(result);
25         }
26         i++;
27     }
28 }
```

Test Input

```
23+
```

Output

```
Result = 5
```

Pending manual review



stdin input (optional)

Output

Inserted element = 10



Pending manual review

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *next;
7 };
8
9 int main() {
10     struct Node *head = NULL;
11     struct Node *newNode;
12
13     newNode = (struct Node *)malloc(sizeof(struct Node));
14
15     newNode->data = 10;
16     newNode->next = head;
17     head = newNode;
18
19     printf("Inserted element = %d", head->data);
20
21     free(newNode);
22     return 0;
23 }
```

Submitted



As a debugging team find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

Test Input

stdin input (optional)

Output

Reversed List: 30 20 10



Pending manual review

```
1 #include <stdio.h>
2
3 struct Node {
4     int data;
5     struct Node *next;
6 };
7
8 int main() {
9     struct Node n1, n2, n3;
10    struct Node *head, *current, *prev, *next;
11
12    n1.data = 10;
13    n1.next = &n2;
14    n2.data = 20;
15    n2.next = &n3;
16    n3.data = 30;
17    n3.next = NULL;
18
19    head = &n1;
20    current = head;
21    prev = NULL;
22
23    while (current != NULL) {
24        next = current->next;
25        current->next = prev;
26        prev = current;
```



🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 int stack[5] = {10, 20, 30};
4 int top = 2;
5
6 int pop() {
7     if (top == -1) {
8         printf("Underflow\n");
9         return -1;
10    }
11
12    return stack[top--];
13 }
14
15 int main() {
16     printf("Deleted element = %d", pop());
17     return 0;
18 }
```

✓ Submitted

Test Input

stdin input (optional)

Output

Deleted element = 30

🕒 Pending manual review



Your Fixed Code

```
1 #include <stdio.h>
2
3 struct Node {
4     int data;
5     struct Node *next;
6 };
7
8 int main() {
9     struct Node n1, n2, n3;
10    struct Node *head = &n1;
11    struct Node *temp;
12
13    n1.data = 10;
14    n1.next = &n2;
15    n2.data = 20;
16    n2.next = &n3;
17    n3.data = 30;
18    n3.next = NULL;
19
20    temp = head;
21
22    while (temp != NULL) {
23        printf("%d ", temp->data);
24        temp = temp->next;
25    }
26
27    return 0;
```

Test Input

stdin input (optional)

Output

10 20 30

🕒 Pending manual review


```

2
3 char stack[20];
4 int top = -1;
5
6 char pop() {
7     return stack[top--];
8 }
9
10 int check(char ch) {
11     if (top == -1)
12         return 0;
13
14     if ((ch == ']' && stack[top] == '[') ||
15         (ch == '}' && stack[top] == '{') ||
16         (ch == ')' && stack[top] == '(')) {
17         pop();
18         return 1;
19     }
20
21     return 0;
22 }
23
24 int main() {
25     stack[++top] = '[';
26
27     if (check(']'))
28         printf("Bracket matched");
29     else
30         printf("Bracket not matched");
31

```

Output

```
Bracket matched
```

 Pending manual review

SIMATS Online Programming Lab

DS PROGRAMMING



GANGINENI Koushik
192524238RB



```

1 #include <stdio.h>
2
3 char stack[20];
4 int top = -1;
5
6 char pop() {
7     return stack[top--];
8 }
9
10 int main() {
11     char ch = ')';
12     int j = 0;
13     char postfix[20];
14
15     if (ch == ')') {
16         while (top != -1 && stack[top] != '(') {
17             postfix[j++] = pop();
18         }
19
20         if (top != -1)
21             pop(); // Remove '('
22     }
23
24     postfix[j] = '\0';
25
26     printf("Postfix = %s", postfix);
27
28     return 0;
29 }

```

stdin input (optional)

Output

```
Postfix =
```

 Pending manual review

🔍 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 int stack[5];
4 int top;
5
6 void init() {
7     top = -1;
8 }
9
10 int main() {
11     init();
12     printf("Stack initialized successfully");
13     return 0;
14 }
```

Test Input

stdin input (optional)

Output

Stack initialized successfully

🕒 Pending manual review

✅ Submitted

SIMATS Online Programming Lab

DS PROGRAMMING



GANGINENI KOUSHIK
192524238RB



🔍 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 int stack[5] = {10, 20, 30};
4 int top = 2;
5
6 int pop() {
7     if (top == -1) {
8         printf("Underflow\n");
9         return -1;
10    }
11
12    return stack[top--];
13 }
14
15 int main() {
16     int x = pop();
17
18     if (x != -1)
19         printf("Deleted element = %d", x);
20
21     return 0;
22 }
```

Test Input

Stack = 10 20 30

Output

Deleted element = 30

🕒 Pending manual review

✅ Submitted



```
1 #include <stdio.h>
2
3 int binarySearch(int arr[], int n, int key) {
4     int low = 0, high = n - 1;
5
6     while (low <= high) {
7         int mid = (low + high) / 2;
8
9         if (arr[mid] == key)
10             return mid;
11         else if (arr[mid] < key)
12             low = mid + 1;
13         else
14             high = mid - 1;
15     }
16
17     return -1;
18 }
19
20 int main() {
21     int arr[] = {10, 20, 30, 40, 50};
22     int key = 30;
23
24     int result = binarySearch(arr, 5, key);
25
26     if (result != -1)
27         printf("Element found at index %d", result);
28     else
29         printf("Element not found");
30 }
```

Array: 10 20 30 40 50
Key: 30

Output

Element found at index 2

⌚ Pending manual review



🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *next;
7 };
8
9 int main() {
10     struct Node *head = NULL;
11     struct Node *newNode;
12
13     newNode = (struct Node *)malloc(sizeof(struct Node));
14
15     newNode->data = 10;
16     newNode->next = head;
17     head = newNode;
18
19     printf("Inserted element = %d", head->data);
20
21     free(newNode);
22     return 0;
23 }
```

Test Input

stdin input (optional)

Output

Inserted element = 10

⌚ Pending manual review



```
peek(stack);  
return stack[top+1]
```

🔧 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>  
2  
3 int stack[5] = {10, 20, 30};  
4 int top = 2;  
5  
6 int peek() {  
7     if (top == -1) {  
8         printf("Underflow");  
9         return -1;  
10    }  
11    return stack[top];  
12 }  
13  
14 int main() {  
15     printf("Top element = %d", peek());  
16     return 0;  
17 }
```

Test Input

stdin input (optional)

Output

Top element = 30

🕒 Pending manual review